

maximatch build unknown	Extends <code>multimatch()</code> with support for filter functions and regular expressions Install \$ npm install --save maximatch	Usage <pre>var maximatch = require('maximatch'); maximatch(['unicorn', 'cake', 'rainbows'], ['*', '!cake']); //=> ['unicorn', 'rainbows'] maximatch(['unicorn', 'cake', 'rainbows'], function(path) { return path.length > 4; }); //=> ['unicorn', 'rainbows']</pre>	<pre>maximatch(['unicorn', 'cake', 'rainbows'], /^[^k]+\$//); //=> ['unicorn', 'rainbows'] maximatch(['unicorn', 'cake', 'rainbows'], [function(path) { return path.charAt(0) === 'u'; }, /w/]); //=> ['unicorn', 'rainbows']</pre> See the tests for more usage examples and expected matches.
1 8 MIT © Sindre Sorhus, Jon Schlinkert, Tim Kendrick License See globby if you need to match against the filesystem instead of a list. Related • <code>i</code> at the beginning of a pattern will negate the match	2 7 Just a quick overview. • <code>*</code> matches any number of characters, but not <code>/</code> • <code>?</code> matches a single character, but not <code>/</code> • <code>**</code> matches any number of characters, including <code>/</code>, as long as it's the only thing in a path part • <code>{ }</code> allows for a comma-separated list of "or" expressions Globbing patterns	3 6 Positive patterns (e.g. <code>foo</code> or <code>*</code>) add to the results, while negative patterns (e.g. <code>!foo</code>) subtract from the results. Therefore a lone negation (e.g. <code>['!foo']</code>) will never match anything – use <code>['*', '!', '!'foo']</code> instead. How multiple patterns work	4 5 Same as <code>minimatch.match()</code> except for pattern also accepting a filter function, a regular expression, or an array that can contain globs, filter functions and regular expressions. API <pre>var results = maximatch(paths, patterns);</pre> The return value is an array of matching paths.
6 16	10 15	11 14	12 13